

Reproducible Data Workflow: A Foundation Analysis of the Titanic Survival Dataset

Module Summary — AI Programming Foundations Project

Author: Tarie Nosworthy

Overview

This report documents a complete, reproducible data workflow built on the Titanic passenger dataset (Kaggle, <https://www.kaggle.com/c/titanic>). The workflow ingests the raw CSV, applies documented cleaning functions, performs a reusable exploratory analysis, and produces four labeled visualizations that surface the dataset's dominant survival patterns. No predictive model is trained; the objective is a clean, auditable foundation on which later machine-learning, deep-learning, and agentic projects can build.

Dataset Description

The Titanic dataset is a passenger manifest of 891 rows and 12 columns describing individuals aboard the RMS Titanic. Each row represents one passenger, with a binary *Survived* outcome (0 = did not survive, 1 = survived) and features including passenger class (*Pclass*), *Sex*, *Age*, number of siblings/spouses aboard (*SibSp*), number of parents/children aboard (*Parch*), ticket *Fare*, and port of embarkation (*Embarked*). The analysis focuses on the relationship between survival and the three features carrying the strongest signal: sex, class, and age. Three columns contained missing values on ingestion — *Age* (177 missing, ~20%), *Cabin* (687 missing, ~77%), and *Embarked* (2 missing) — and each was handled explicitly during cleaning.

Workflow Description

Ingestion. The raw CSV is loaded with pandas into a dataframe, and an untouched copy is preserved so that every downstream transformation remains traceable to its source. Separating raw from working data is a basic reproducibility safeguard consistent with open, transparent workflow practice (Danchev, 2022).

Cleaning. Three documented functions perform the cleaning stage: a diagnostic that quantifies missingness, an imputation routine for *Age* and *Embarked*, and a pruning routine that removes the near-empty *Cabin* column and any duplicate rows. Structuring each variable so that it is analysis-ready reflects established tidy-data principles (Wickham, 2014).

Exploratory analysis. A single parameterized function, *survival_summary*, computes counts and survival rates for any grouping column, and is reused across sex, class, and port of embarkation.

Visualizations. Four Matplotlib/Seaborn figures — each with a title and labeled axes — present survival by sex, survival by class, the age distribution by outcome, and the numeric correlation structure.

Summary. A written interpretation in the notebook and in this report consolidates the findings, assumptions, and limitations.

Key Decisions and Assumptions

Cleaning choices. *Age* was imputed using the median within each *Pclass*×*Sex* group rather than a single global median. *Age* varies systematically with class and sex, so a grouped median preserves that structure, and the median resists distortion from a small number of much older passengers. *Embarked* (two missing values) was filled with the overall mode, the lowest-risk choice given how few values were absent. *Cabin* was dropped rather than imputed: with roughly 77% of values missing, imputation would fabricate more data than it preserves. Handling missing data through transparent, documented rules rather than silent defaults supports reproducible analysis (Danchev, 2022).

EDA focus. The exploratory function targets grouped survival rates because the research interest is which passenger characteristics were associated with survival; a parameterized design keeps the aggregation logic consistent across every grouping.

Visualization intent. Figure 1 isolates the single strongest predictor (sex); Figure 2 shows the class gradient; Figure 3 examines whether age adds signal; and Figure 4 checks for redundancy among numeric features before any future modeling.

Results and Interpretation

The dataset's survival structure is strong and consistent. As shown in **Figure 1**, about 74% of women survived versus roughly 19% of men, making sex the single most informative feature. **Figure 2** shows survival falling monotonically with class — approximately 63% in first class, 47% in second, and 24% in third — reflecting differences in cabin location, deck access, and evacuation priority. **Figure 3** shows heavily overlapping age distributions for survivors and non-survivors, indicating that age is a weaker standalone predictor, with a modest advantage visible only at the youngest ages. **Figure 4** confirms that *Fare* correlates positively with survival (~0.26) and negatively with *Pclass*, consistent with higher fares in higher classes, so the two encode overlapping information. Taken together, the results point to an interaction between social priority and structural advantage as the dominant driver of survival in this record.

Responsible Practice (Bias and Data Quality)

Several cleaning decisions could introduce bias if applied without scrutiny. Grouped-median imputation of *Age* fills about a fifth of that column and compresses within-group variance, concentrating imputed values at the group medians; this is visible as a spike in Figure 3 and could bias any age-based conclusion toward those central values. Dropping *Cabin* avoids fabricating data but discards whatever deck-location signal it carried, which may itself correlate with survival. More fundamentally, the dataset records only passengers with manifest entries, so findings describe this historical event rather than a general population. To reduce these risks, imputation was kept grouped and transparent rather than global, the near-empty column was dropped rather than guessed, and every decision was documented so a reviewer can audit its downstream effect

— an approach aligned with reproducible, ethically aware data practice (Danchev, 2022).

Reproducibility

Another analyst can rerun this work exactly. All dependencies are pinned in *requirements.txt*, generated with `pip freeze > requirements.txt` and installed with `pip install -r requirements.txt`. A single random seed is set once at the top of the notebook, and the raw dataframe is copied before any transformation so the source is never mutated. Version control is handled through Git: the repository contains multiple commits tracking each task and at least one development branch beyond *main*, providing a transparent history of how the workflow was constructed. Pinned environments and explicit version control are core to reproducible computational research (Danchev, 2022).

References

- Danchev, V. (2022). Reproducible data science with Python: An open learning resource. *Journal of Open Source Education*, 5(56), 156. <https://doi.org/10.21105/jose.00156>
- Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, 59(10), 1–23. <https://doi.org/10.18637/jss.v059.i10>